Design&Build 2025

Team 23

International School

CONTENTS

01 Hardware Modules

02 Software Works

03 Team Members

目 录

01

Hardware Modules of Mobile Robot

Core Modules

STM32F446RE

It is responsible for real-time sensor processing, motor control, and low-level communication with the host, serving as the core that connects various modules.

AT8236 Motor Driver

It controls the 520 gear encoder DC motor to achieve precise speed and direction control.

520 Gear Encoder DC Motor

It acquires instructions from the motor driver and controls the rotation of the wheels.

Configuration of Functional Modules

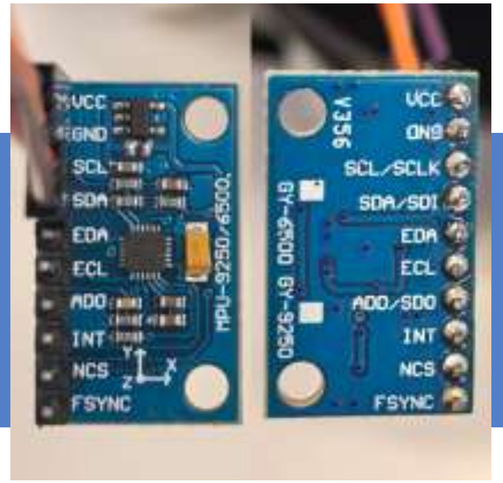
HC-04 Bluetooth Module

It enables wireless communication between the robot and the host (i.e., mobile phone, computer), supporting bidirectional data transmission.

- **Mobile Phone → Car:** Send instructions via mobile phone APP to remotely control the car's movement.
- **Car → Mobile Phone:** Send laser radar scanning data and MPU module data (x, y, θ) to the mobile phone via Bluetooth.

SLAMTEC C1 Laser Scanning Ranging Radar

It is used for environment scanning and obstacle detection during maze navigation, providing raw data for the SLAM algorithm to build maps and plan paths.

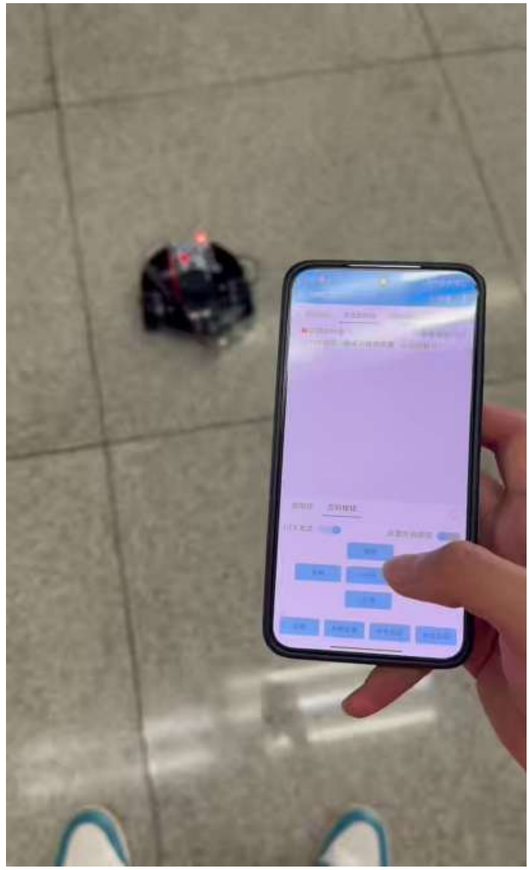


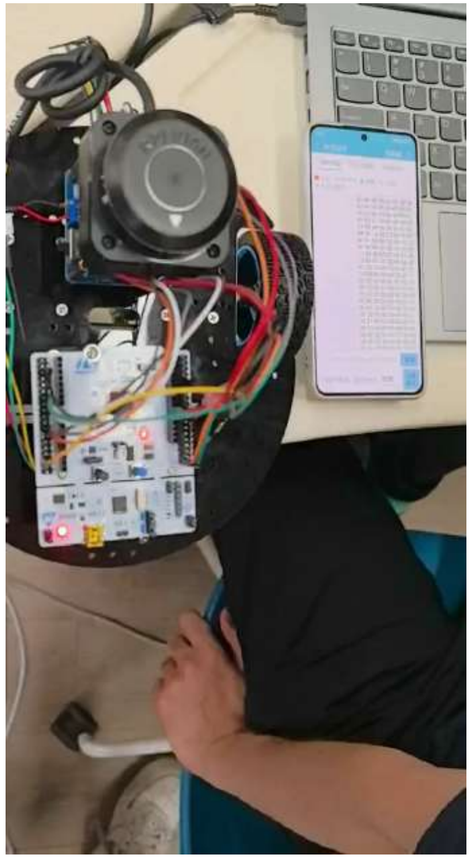
MPU6500 Six-Axis Inertial Measurement Unit

It compensates for the odometer error of the motor encoder, calculates the car's displacement (x, y, θ) by combining gyroscope and accelerometer data, and assists the navigation algorithm in achieving precise positioning.

The foundation obtained in the previous stage

In the work of the first part, the mobile robot has initially realized the function of changing its traveling direction according to Bluetooth remote control commands and transmitting radar data back.





Technical Pain Points in Hardware-Software Integration

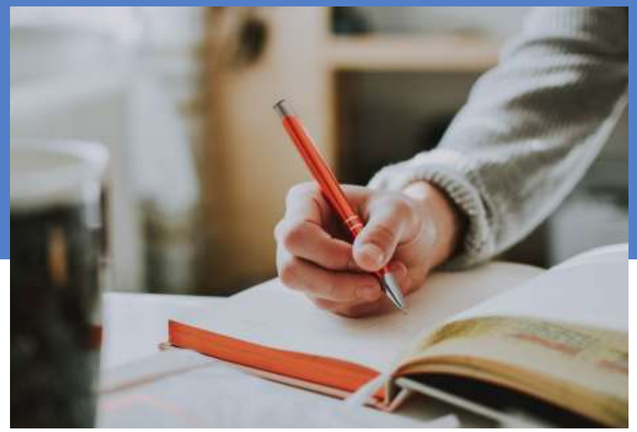
Motion Control Precision Defect

Zero drift of MPU (Inertial Measurement Unit) yaw angle results in attitude calculation errors. This directly affects the angular precision of the car’ s 90° left/right turns and 180° U-turns. Without a closed-loop control mechanism, the unbalanced speed of left and right wheel motors causes straight-line deviation, failing to meet the motion benchmark requirements for upper computer mapping.

Data Transmission Timing Confusion

There is no clear transmission logic between radar scanning data and odometer data. During motion, radar data is prone to spatial ambiguity due to dynamic changes in the carrier’ s position. The upper computer cannot accurately associate "motion status - perception data", which impairs the efficiency of mapping data fusion.

Core Objectives



Motion Control Layer

Build a hardware control chain of "attitude perception - closed-loop control - step execution". Achieve controllable steering angle error, stable straight-line trajectory, and quantitative step stop to meet the discrete motion requirements during mapping.

Data Transmission Layer

Design a transmission trigger strategy based on motion status. Realize data classification and synchronization with identification frames to ensure the real-time performance of odometers and the accuracy of radar data, providing reliable measurement and control data support for the upper computer.

Motion Module Optimization

MPU Yaw Angle Zero Drift Correction

Technical Logic: As the heading reference, MPU’ s zero drift causes cumulative deviation between the carrier coordinate system and geographic coordinate system. Hardware calibration or algorithm compensation (e.g., zero bias compensation algorithm) is required to correct raw attitude data, ensuring the stability of the yaw angle measurement reference.

Core Value: Provides a reliable attitude feedback source for subsequent heading control, directly ensuring the measurement and execution accuracy of steering angles (90°/180°), and solves the core problem of "mismatch between command angle and actual angle".

Dual-Loop PID Control Architecture

Control Logic (Dual-Loop Collaboration):

1. Inner Loop (Motor Speed Closed Loop): Takes the speed difference of left/right wheel motors as the control variable, uses a PID regulator to suppress speed fluctuations in real time, eliminates speed imbalance caused by inconsistent motor parameters (e.g., armature resistance difference, load disturbance), and ensures the foundation for straight-line travel from the execution end.
2. Outer Loop (Heading Closed Loop): Takes the corrected yaw angle of MPU as the feedback variable, uses the deviation between actual heading and desired heading as the PID input, dynamically adjusts the speed difference of left/right wheels, and realizes real-time heading correction.

Motion Module Optimization – Stepping Control Design

Function Definition

Based on motor pulse counting or odometer feedback, the car is enabled to travel straight at a preset step size (discretized displacement). When the displacement reaches the set threshold, automatic stop is triggered, forming a periodic motion mode of "displacement - stop".

Technical Relevance (Collaboration with Upper Computer Mapping)

1. Timing Matching: The upper computer’ s SLAM (Simultaneous Localization and Mapping) algorithm requires timing collaboration of "carrier stationary - environment scanning". Stepping control achieves precise displacement control through quantized step sizes, ensuring the car’ s spatial position is fixed during radar scanning and avoiding spatial registration errors of scanning data caused by motion.
2. Data Association: Stepping stop nodes can serve as the timestamp reference for the upper computer’ s "odometer data - radar data", improving the spatiotemporal correlation of the two types of data and providing reliable timing anchor points for subsequent map fusion.

Data Transmission Strategy

Transmission Timing Allocation Logic

1. Odometer Data: Adopts a "real-time transmission in motion state" mechanism. Establishes a real-time data stream between the upper computer and the car via the UART serial interface, with transmitted content including cumulative displacement and motion direction state quantities, meeting the upper computer’ s demand for real-time monitoring of the car’ s motion status.

2. Radar Scanning Data: Adopts a "triggered transmission in stationary state" mechanism. Radar data collection and upload are only initiated when the car completes stepping motion and is in a stationary state, avoiding environmental perception errors caused by the coupling between radar beam scanning angle and car position during motion.

Data Transmission Optimization

Design Purpose

Solve the problem of "data frame ambiguity": In original transmission, odometer data includes real-time data in motion state and stepping stop data in stationary state, while radar data includes data in start/stop phases. Identification frames are required to achieve accurate binding between data type and motion state, ensuring the uniqueness of upper computer data parsing.

Specific Definition of Identification Frames

1. Radar Data Identification Frames:

- Start Identifier: "START SCAN" (serves as the synchronization header of radar scanning data frames, triggering the initialization of the upper computer’ s radar data reception buffer);
- Stop Identifier: "END SCAN" (serves as the terminator of radar scanning data frames, triggering the upper computer’ s radar data processing process);

2. Odometer Data Identification Frames (triggered only when stepping stops):

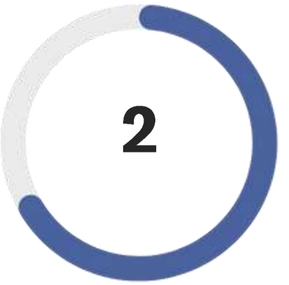
- Start Identifier: "START ODOM" (identifies that the current odometer data is the state data after stepping stops, bound to the start node of upper computer mapping);
- Stop Identifier: "END ODOM" (identifies the completion of odometer data transmission corresponding to this stepping, triggering the associated update of upper computer odometer data and map coordinates)

Summary of Modification Value & Pending Technical Details

Control Layer

Build a hardware control closed-loop of "MPU attitude perception - dual-loop PID control - stepping execution". Enhance the controllability and stability of the carrier's motion, which is in line with the hierarchical architecture of "perception - decision - execution" in telecommunication measurement and control systems.

Transmission Layer



Design a data stream management mechanism of "time-sharing transmission + identification frame synchronization". This enables the upper computer to clearly recognize the car's motion status while improving the efficiency of data interaction between the upper computer and the car, laying a foundation for the hardware-level support of subsequent SLAM algorithms.

Cooperativity

The motion control logic and data transmission strategy are deeply matched, realizing the triple synergy of "motion status - data type - mapping demand" and reducing system-level errors.

02

Software Works of Mobile Robot

Part 1: Navigation Algorithm Logic for Mobile Robot

02

Phase 1: Exploration

Moving Toward the Goal

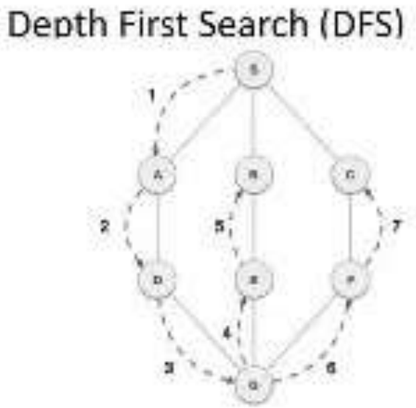
The system adopts a Greedy & Depth-First Search (DFS) algorithm with a backtracking mechanism to explore the unknown environment efficiently.

step1:
From the current position, the robot probes neighboring cells at a fixed distance (step_size = 5). Each grid corresponds to a discretized section of the world coordinate system.

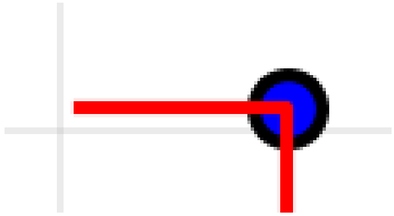
step2:
Path Validation: Each candidate step is validated for safety before movement:
– is_safe_point: Ensures no collision with any known obstacle (BLK).
– is_inside_maze: Checks that the candidate point is within the physical maze boundaries.

step3：
Greedy Decision: Among all “safe” neighbors, the next exploration target is chosen based on the minimum Chebyshev distance (cheb) to the final destination (exit point).
Backtracking: If the robot encounters a dead end (no safe neighbor), it backtracks using a stack (stack_exp) that records the exploration path.

The algorithm continues until the exit point is reached or all reachable spaces are explored.



Navigation Algorithm



```
阶段一: 从起点 (7, 1) 前往终点 (1, 7)...
  -> 找到路径安全的邻居: (30, 5)
  -> 找到路径安全的邻居: (35, 10)
  -> 找到路径安全的邻居: (25, 5)
  -> 找到路径安全的邻居: (25, 10)
  -> 路径到 (20, 10) 在 (21, 10) 处无效 (安全或边界问题)。
```

```python
# 3. 寻找所有有效的邻居候选点（逻辑源自explore）
neighbors = [] # 栅格
step_size = 5  # 探索步长
for dx, dy in [(-step_size, 0), (step_size, 0), (0, -step_size), (0, step_size)]:
    ng = (current_grid_real[0] + dx, current_grid_real[1] + dy)
    if ng in visited: continue
    if self.occ.get(ng, UNK) == BLK: continue

    # --- 2. 核心修改：检查从当前点到邻居的整条路径 ---
    is_path_valid = True
    # 获取步进方向（-1, 0, 或 1）
    step_x = int(np.sign(dx))
    step_y = int(np.sign(dy))

    # 逐一检查路径上的每一个格子
    # 范围从1到step_size，包含起点和终点之间的所有格子
    for i in range(1, step_size + 1):
        check_grid = (current_grid_real[0] + i * step_x, current_grid_real[1] + i * step_y)

        # 【新增】将检查点从栅格坐标转换为世界坐标，以用于边界检查
        check_world_x, check_world_y = self.robot.grid_to_world(*check_grid)

        # 【核心】执行双重检查：
        # 1. is_safe_point: 检查是否撞到已知障碍物
        # 2. is_inside_maze: 检查是否超出物理边界
        if not self.is_safe_point(check_grid, safety_margin=1) or \
           not self.maze.is_inside_maze(check_world_x, check_world_y, margin=0.2):

            # 只要任一检查失败，整条路径就视为无效
            is_path_valid = False
            print(f"  -> 路径到 {ng} 在 {check_grid} 处无效 (安全或边界问题)。")
            break # 无需再检查此路径的其余部分

    # --- 3. 只有整条路径都安全的邻居，才能被选中 ---
    if is_path_valid:
        print(f"  -> 找到路径安全的邻居: {ng}")
```

Phase 2: Return

Following the path back

step 1

The exploration path stack (stack_exp) is reversed (stack[::-1]) to produce a path from the goal back to the origin.

step 2

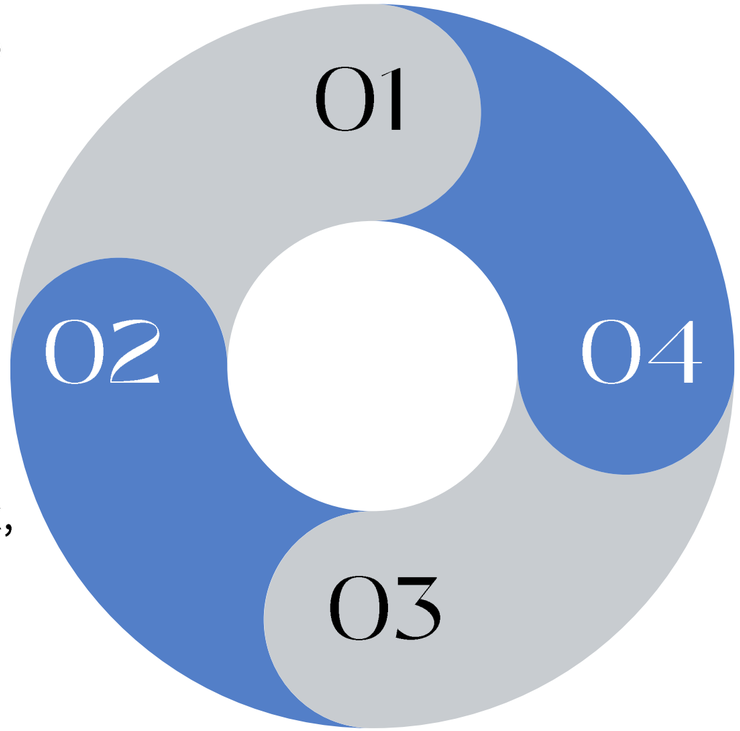
The reversed path is treated as a list of waypoints.

step 3

The robot sequentially follows these waypoints to navigate back, applying motion commands to correct its position at each step.

step 4

This ensures a deterministic, safe, and efficient return route without re-computation.



Part 2: Command Control Logic for Mobile Robot

02

The system adopts a Greedy Depth-First Search (DFS) algorithm with a backtracking mechanism to explore the unknown environment efficiently.

Step 1: Decision — Compute Direction Deviation

Concept: The robot continuously compares its target orientation with its current heading angle.

- The target world angle (target_angle_world) is computed based on the next waypoint in global coordinates.
- The robot’s absolute heading (self.robot.theta) is obtained from the odometry system.
- The angular difference (angle_diff) = target_angle_world – self.robot.theta determines how much rotation is required before moving forward.

Step 2: Mapping — Angle Difference to Discrete Command

Concept: Based on the angular deviation, motion commands are quantized into a small set of discrete actions.

Logical Mapping (if/elif structure):

- Small angular deviation (target roughly ahead): Send command ‘1’ (Move Forward).
- Target to the Left: Send ‘2’ (Turn Left), delay for rotation, then send ‘1’ (Move Forward).
- Target to the Right: Send ‘3’ (Turn Right), delay, then send ‘1’ (Move Forward).
- Target Behind: Send ‘4’ (Turn Around/Reverse), delay, then send ‘1’ (Move Forward).

This ensures smooth transitions between rotational and translational motions while maintaining real-time responsiveness.

Step 3: Execution — Command Transmission via Bluetooth

Concept: The robot’s onboard controller communicates with its motor system using Bluetooth serial communication.

- The method send_command() (from robot1.py) encodes each control command (e.g., ‘1’) and transmits it through a serial port.
- This modular approach separates motion control from navigation logic, allowing future integration with other wireless protocols or onboard computation units.

Part 3: Mapping Logic (SLAM — Simultaneous Localization and Mapping)

02

Step 1: Data Acquisition — Lidar and Odometry Fusion

Concept: The system fuses sensor data from Lidar scans and odometry readings.

- The scan() method calls _receive_frame() to obtain raw Lidar data points (angle_deg, distance).
- Each scan point is transformed from the robot’ s local frame to world coordinates using the current pose (self.x, self.y, self.theta).
- The transformed coordinates (wx_hit, wy_hit) represent the absolute positions of detected obstacles in the environment.
- All obstacle points are stored in a list self.hits for subsequent processing.

Step 2: Map Update — Occupancy Grid Maintenance

Concept: The occupancy map is implemented as a sparse dictionary for efficient updates and memory use.

- The update_occ() function iterates through all world-coordinate obstacle points in self.hits.
- Each point is converted to grid coordinates (gx, gy) via world_to_grid(wx, wy).
- The grid cell (gx, gy) is marked as occupied: self.occ[(gx, gy)] = BLK (Blocked).
- Over time, the occupancy map becomes denser, reflecting more of the explored environment.
- Free spaces can be inferred by ray tracing between the robot position and detected obstacles.

Step 3: Integration with Navigation

The updated map directly influences the exploration phase of the navigation algorithm.

- The is_safe_point function queries the map to ensure that planned movements do not intersect blocked regions.
- As the map evolves, the robot adapts its exploration behavior dynamically, improving efficiency and collision avoidance.
- This tight coupling between SLAM and path planning creates a closed-loop autonomous navigation system.

Part 4: Bluetooth R/T Logic for DataTransmission System

02

Logical components

Send (Command): Simply encode a Python string (e.g., 1) usingencode(utf-8)and send it via self.connection.write().

Receive (Data): Utilizes a token-based frame synchronization protocol, employing the readline() function to cyclically wait for specific markers.

(Frame synchronization:The process of the receiverdistinguishing different information marked by the sender usingspecial codes on the information stream)

Protocol process

Frame mileage

wait for START_ODOM

Continuously read and parse lines starting with Odom

Directly received END_ODOM

Lidar mileage only during exploration

wait for START_SCAN

Continuously read and parse lines starting with lidar

Directly received END_SCAN

In order to avoid confusion in the collected data, which would make it difficult to process, we have chosen an innovative frame-synchronous processing method

Data parsing: Extract key values from data rows using regularsynchroniza frame expressions (re.search)

Odom: mate X=, Y=, Yaw=

Lidar: mate A= (angle), D= (distance)

Demonstration of sending and receiving commands:

```python
if distance_to_target > 5:
    print(f"距离目标 {next_target_grid} 较远 (距离: {distance_to_target})，开始移动...")

    # 计算方向并发送指令 (逻辑与探索时完全一致)
    start_w = (self.robot.x, self.robot.y)
    target_w = self.robot.grid_to_world(*next_target_grid)
    target_angle_world = math.atan2(target_w[1] - start_w[1], target_w[0] - start_w[0])
    angle_diff = target_angle_world - self.robot.theta
    angle_diff = (angle_diff + math.pi) % (2 * math.pi) - math.pi

    if -math.pi / 4 <= angle_diff <= math.pi / 4:
        self.robot.send_command('1') # 前进
        time.sleep(3.0) # 假设前进一格需要1秒
    elif math.pi / 4 < angle_diff < 3 * math.pi / 4:
        self.robot.send_command('2') # 左转
        time.sleep(1.5) # 假设左转90度需要1.5秒
        self.robot.send_command('1') # 前进
        time.sleep(1.0)
    elif -3 * math.pi / 4 < angle_diff < -math.pi / 4:
        self.robot.send_command('3') # 右转
        time.sleep(1.5)
        self.robot.send_command('1') # 前进
        time.sleep(1.0)
    else:
        self.robot.send_command('4') # 后退(掉头)
        time.sleep(1.5)
        self.robot.send_command('1') # 前进
        time.sleep(1.0)

else:
    print(f"距离目标 {next_target_grid} 小于等于5，已到达，无需移动。")
```

```
--- 步骤 1: 发送 '0' 指令以启动数据传输 ---
--- 通过 COM6 发送指令: '0' ---
接收缓冲区已清空，准备接收数据...
--- 通过 COM6 发送指令: '1' ---
当前动作为 = 1，小车开始运动

--- 步骤 2: 等待并接收里程计数据 ---
接收到标志: START_ODOM
接收到数据: odom:{"x":1.011,"y":-0.161,"yaw":350.0}
接收到标志: END_ODOM

>>> 成功接收到里程计数据 <<<
[
  {
    "x": 1.011,
    "y": -0.161,
    "yaw": 350.0
  }
]
里程计数据解算成功: World Pose=[X:0.00, Y:0.00, Yaw:80.00]

--- 步骤 3: 等待并接收雷达扫描数据 ---
接收到标志: START_SCAN
接收到标志: odom:{"x":1.073,"y":-0.172,"yaw":349.7}
接收到标志: Lidar: A=212.41, D=16.92, Q=84
接收到标志: odom:{"x":1.124,"y":-0.181,"yaw":349.7}
接收到标志: odom:{"x":1.149,"y":-0.185,"yaw":349.6}
接收到标志: Lidar: A=288.47, D=552.15, Q=115
接收到标志: odom:{"x":1.186,"y":-0.192,"yaw":349.3}
接收到标志: Lidar: A=324.25, D=12.88, Q=127
接收到标志: odom:{"x":1.214,"y":-0.197,"yaw":349.3}
接收到标志: odom:{"x":1.226,"y":-0.199,"yaw":349.3}
接收到标志: Lidar: A=320.73, D=832.53, Q=99
接收到标志: odom:{"x":1.241,"y":-0.202,"yaw":349.2}
```

Part 5 Coordinate Transformation Logic

02

Logical components

- Grid coordinates ⇔ World coordinates
  - world_to_grid(x, y): By dividing by GRID_LEN (0.2m) and rounding down (math.floor), continuous world coordinates are discretised into grid indices
  - grid_to_world(gx, gy): Get the world coordinates of the centre of a grid cell by multiplying by GRID_LEN and adding half a grid (GRID_LEN/2)
- Odometer → World coordinates

  (Main idea: Convert the relative displacement (offset_x_m, offset_y_m) in the car's coordinate system to the world coordinate system)
  - Rotate this displacement using a 2D rotation matrix (based on the current vehicle yaw angle self.theta)
    {2D rotation matrix (a coefficient matrix representing points after rotation in 2D space using the coordinates of points before rotation)}
  - Add the rotated world's displacement (offset_x_world_m, offset_y_world_m) to the initial starting point (self.start_x, self.start_y) to obtain the current absolute world coordinates self.x, self.y
- Lidar (Relative) →World coordinates

  (Main idea: Convert Lidar's (angle_deg, distance) to world coordinates)
  - absolute_angle = robot_yaw + lidar_angle + offsets (The conversion of radar angles is based on odometry angles and the initial robot pose, so adjustments need to be made using robot_yaw and offsets pose)
    Use trigonometric functions to convert polar coordinates (absolute_angle, distance) to Cartesian coordinates and add the robot's current world coordinates (self.x, self.y)

Coordinate Transformation Content Display

```python
if line.startswith('Odom: '):
    match_X = re.search(r'X=([-\d.]+)', line)
    match_Y = re.search(r'Y=([-\d.]+)', line)
    match_Yaw = re.search(r'Yaw=([-\d.]+)', line)
    if match_X and match_Y and match_Yaw:
        offset_x_m = (float(match_X.group(1)) / 100.0)/0.35
        offset_y_m = (float(match_Y.group(1)) / 100.0)/0.35 # 加旋转矩阵，输入offsetx offsety 小车偏航角（绝对坐标系下）

        robot_yaw_world_degrees =self.theta
        robot_yaw_world_rad = math.radians(robot_yaw_world_degrees)
        offset_x_world_m = offset_x_m * math.cos(robot_yaw_world_rad) - offset_y_m * math.sin(robot_yaw_world_rad)
        offset_y_world_m = offset_x_m * math.sin(robot_yaw_world_rad) + offset_y_m * math.cos(robot_yaw_world_rad)

        absolute_x = self.start_x + offset_x_world_m
        absolute_y = self.start_y + offset_y_world_m
        self.x = absolute_x
        self.y = absolute_y

        absolute_yaw = float(match_Yaw.group(1))
        self.theta = absolute_yaw + THETA_ADD
        if self.theta >=360:
            self.theta -=360

        odom_data = [absolute_x, absolute_y, absolute_yaw]
        odom_received_in_frame = True
        print(f"里程计数据接收成功: World Pose=[X:{absolute_x:.2f}, Y:{absolute_y:.2f}, Yaw:{absolute_yaw:.2f}]")
```

03

Introduction of Team Members

Hardware team division of labour：

Wenyun Zhong： The realization of motion and data transmission functions

Shuo Feng： The realization of motion and data transmission functions

Jialu Sun： PPT production, appearance beautification

Hao Lu： PPT production, appearance beautification

Yulin He： Assisting in the debugging of the car

Mingxuan Liu： Assisting in the debugging of the car

Software team division of labour：

Dongting Ge： Core algorithm coding, PPT framework construction

Siyuan Xu： Test function writing, PPT text layout optimisation

Xiaopeng Tan： Test function writing, PPT text layout optimisation

Liying Wu： Test function writing, result video editing

Thank you for your attention!

Team 23